

IIT DELHI MISN LAB RESEARCH INTERNSHIP



Explanation of Weekly Assignments

WEEK 1: The XOR Problem

WEEK 2: Implementation of Auto grad and XOR using
Multi-Layer Perceptron

WEEK 3: CNN from scratch

Presented By: Nandini Mishra

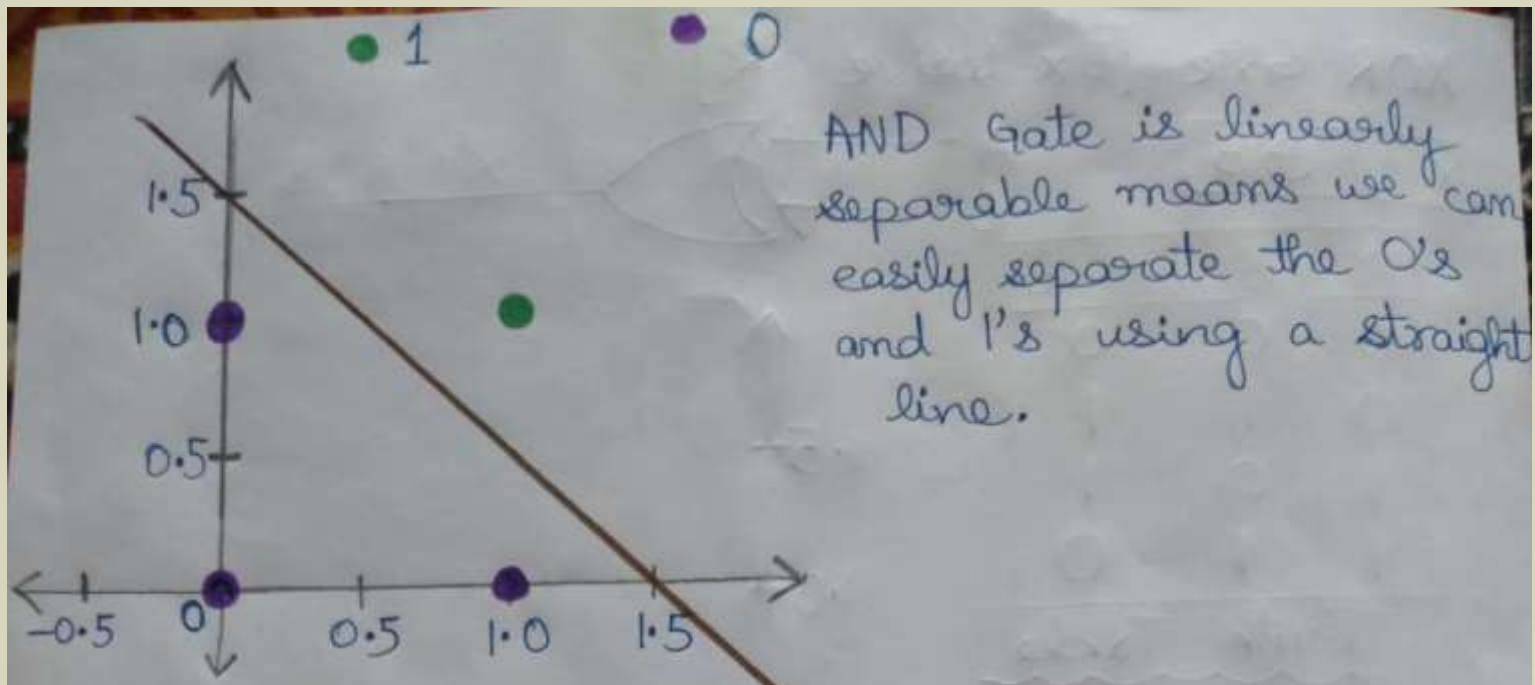
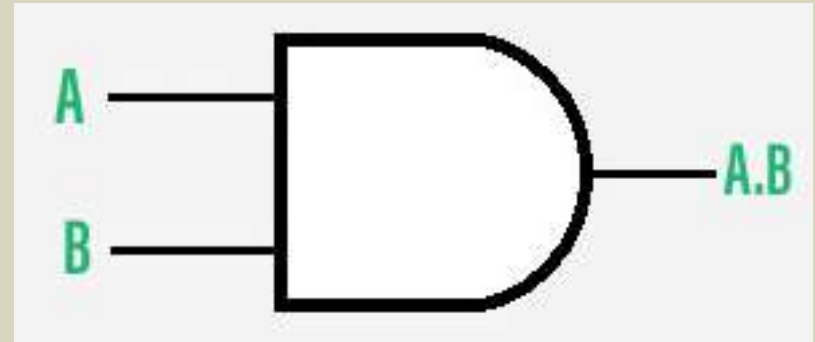
Mentor: Asmita De

Guide: Sandeep Kumar

WEEK 1: THE XOR PROBLEM

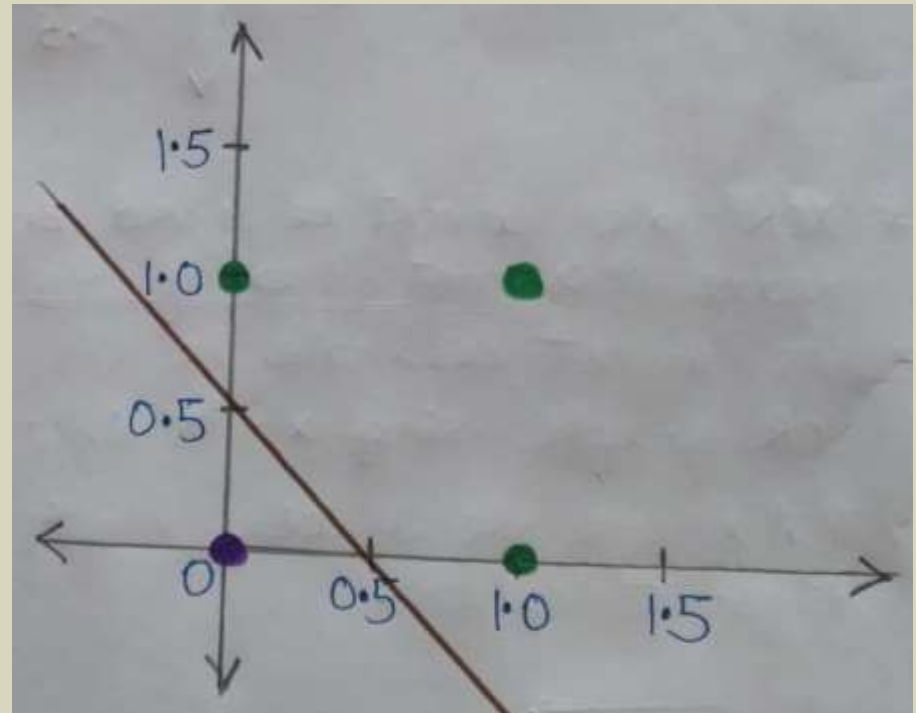
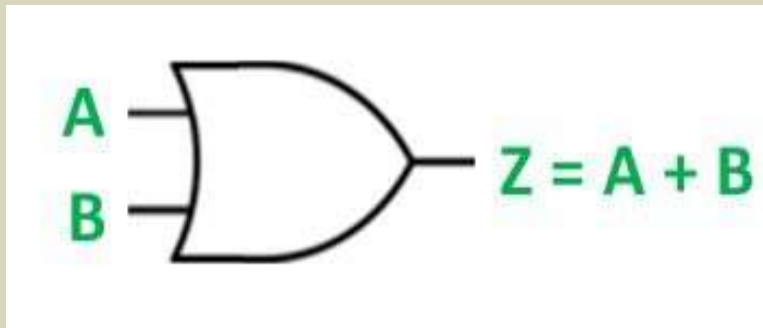
AND GATE

Input		Output
A	B	$Y=A.B$
0	0	0
0	1	0
1	0	0
1	1	1



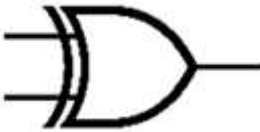
OR GATE

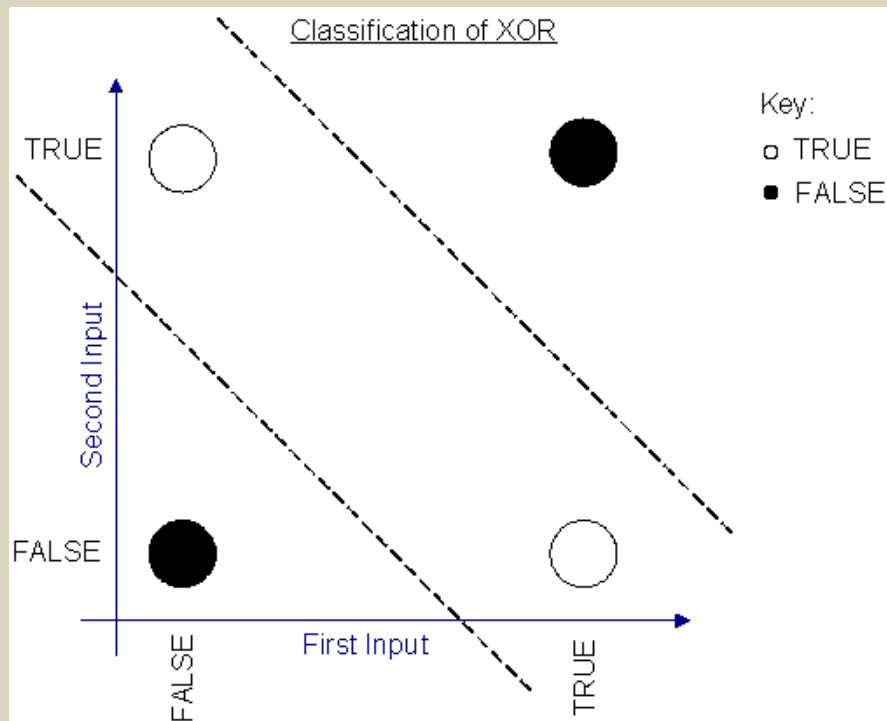
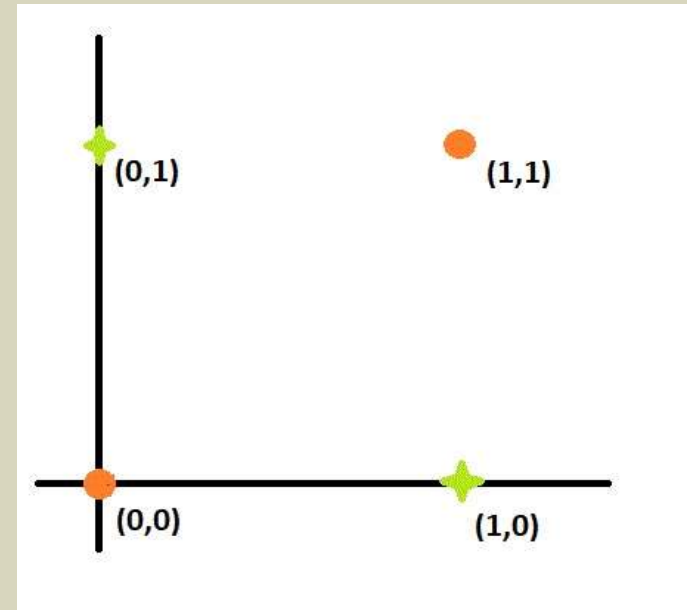
Input		Output
A	B	$Y=A+B$
0	0	0
0	1	1
1	0	1
1	1	1



OR Gate is also linearly separable because it is easy to separate the 0's from 1's using a straight line.

XOR GATE

Symbol	A	B	Q
	0	0	0
	1	0	1
	0	1	1
	1	1	0
$Q = A \oplus B$			

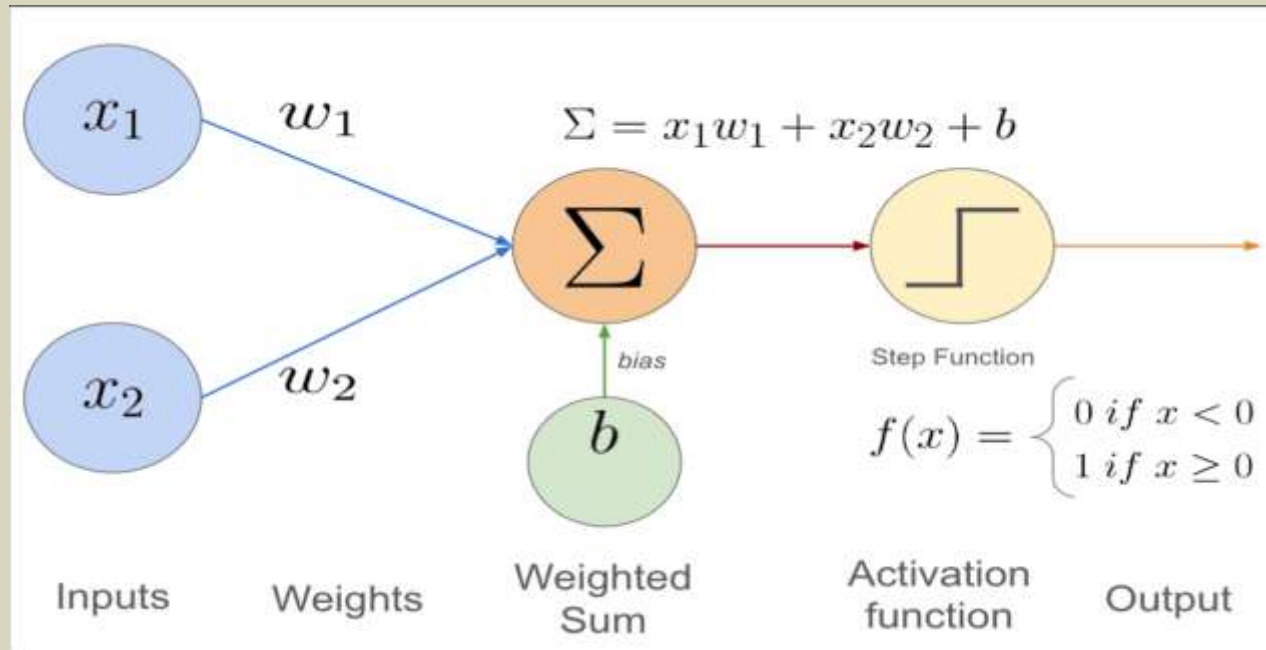


XOR is not linearly separable

One perceptron cannot solve it

Multiple layers can

What is a Perceptron ?



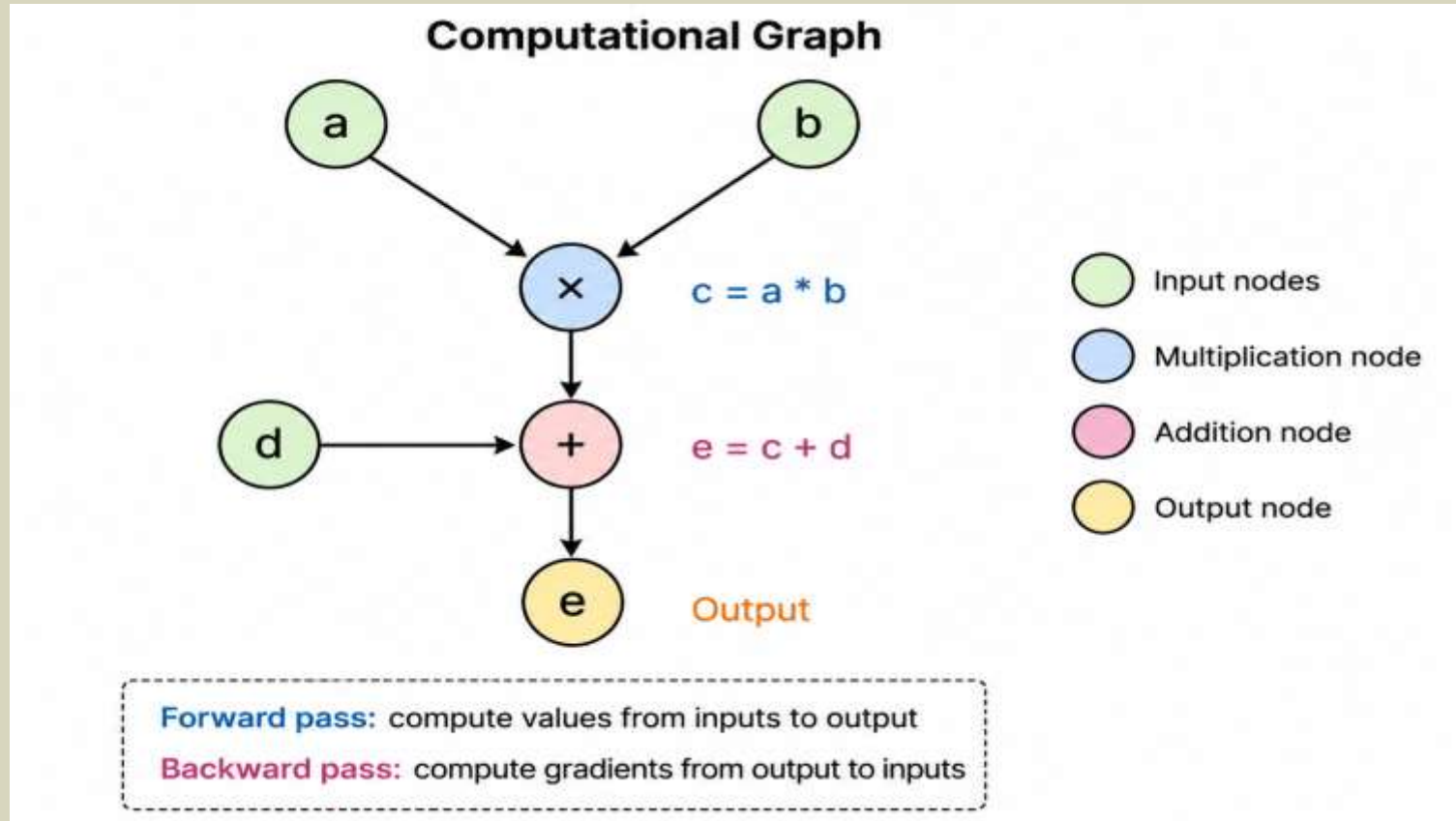
- A Perceptron in Machine Learning is a basic building block of Artificial Neural Network (ANN) which helps in quick decision-making.
- It takes multiple **input** features, **multiplies** them by specific **weights**, **adds a bias**, and passes the sum through an **activation function** to yield a single binary output (e.g., 0 or 1).

WEEK 2 – IMPLEMENTATION OF AUTOGRAD ENGINE

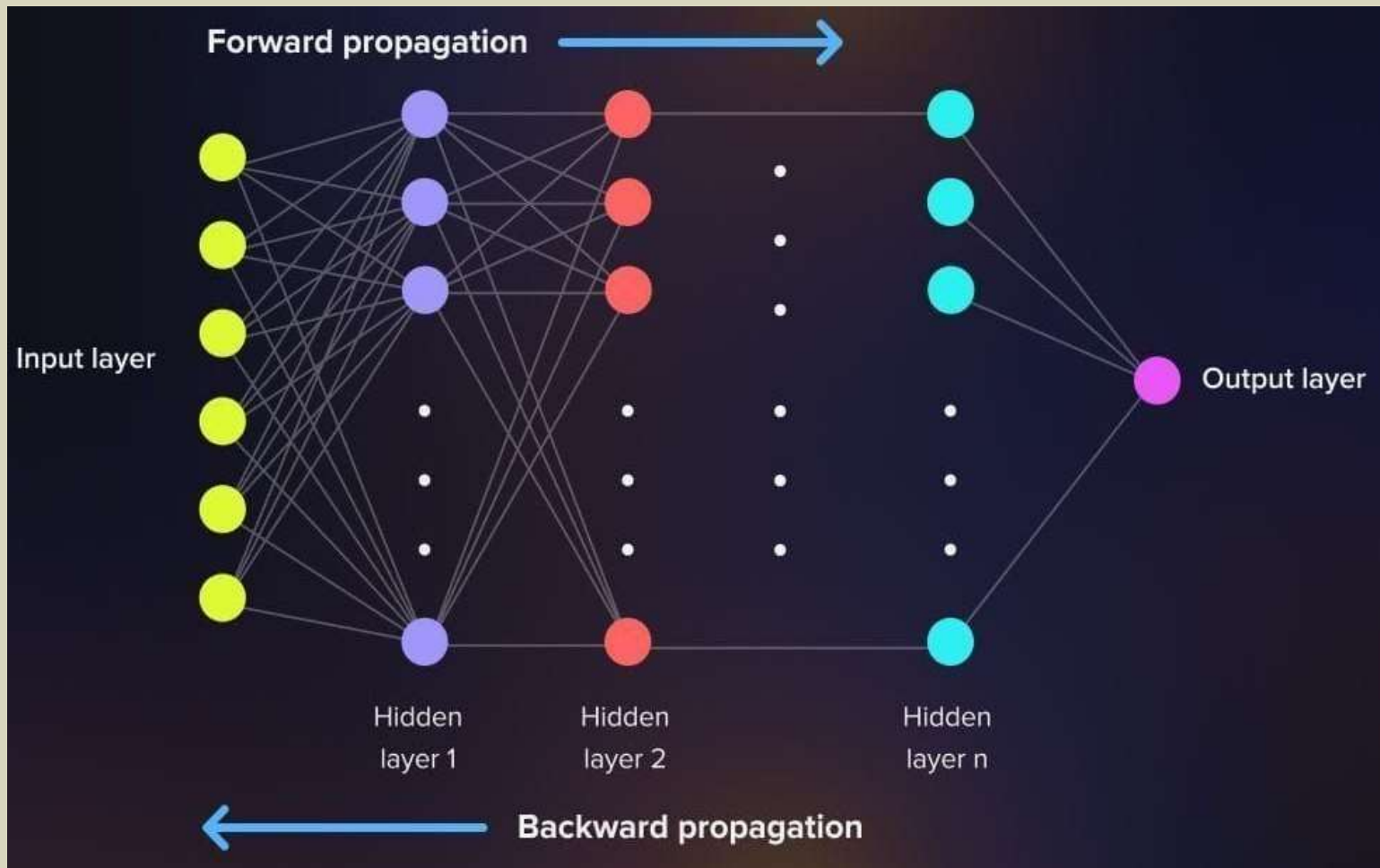
Automatic Differentiation Engine

Objective

- Implement a basic Micro grad and build a computational graph.
- Automatically compute gradients using backpropagation.



Forward and Backward Passes in Neural Networks



Forward pass is for **Prediction** whereas, Backward pass is for **Learning**

BACKPROPAGATION

Implementing the to-do lists

- **Addition backward pass()**

self.grad += out.grad

other.grad += out.grad

- **Multiplication backward pass()**

*self.grad += other.data * out.grad*

*other.grad += self.data * out.grad*

- **Power backward pass()**

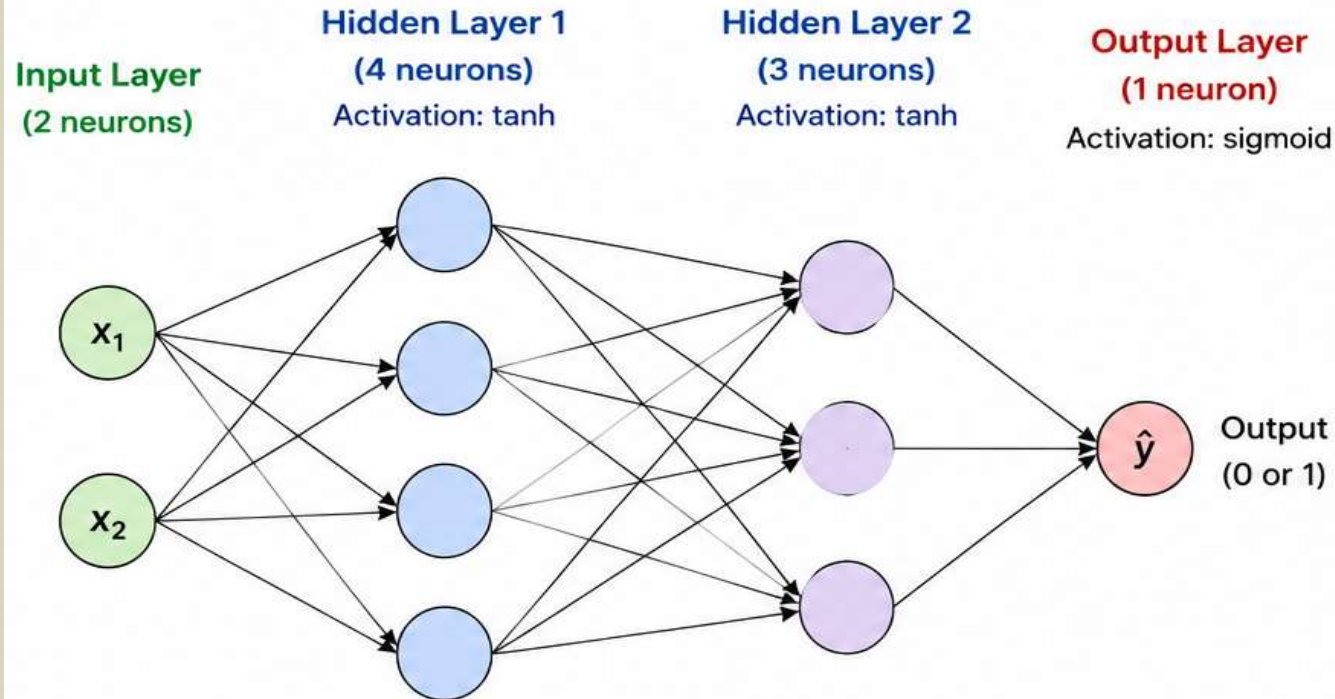
*self.grad += (other * (self.data ** (other - 1))) * out.grad*

- **ReLU { Rectified Linear Unit } backward pass()**

*self.grad += (out.data > 0) * out.grad*

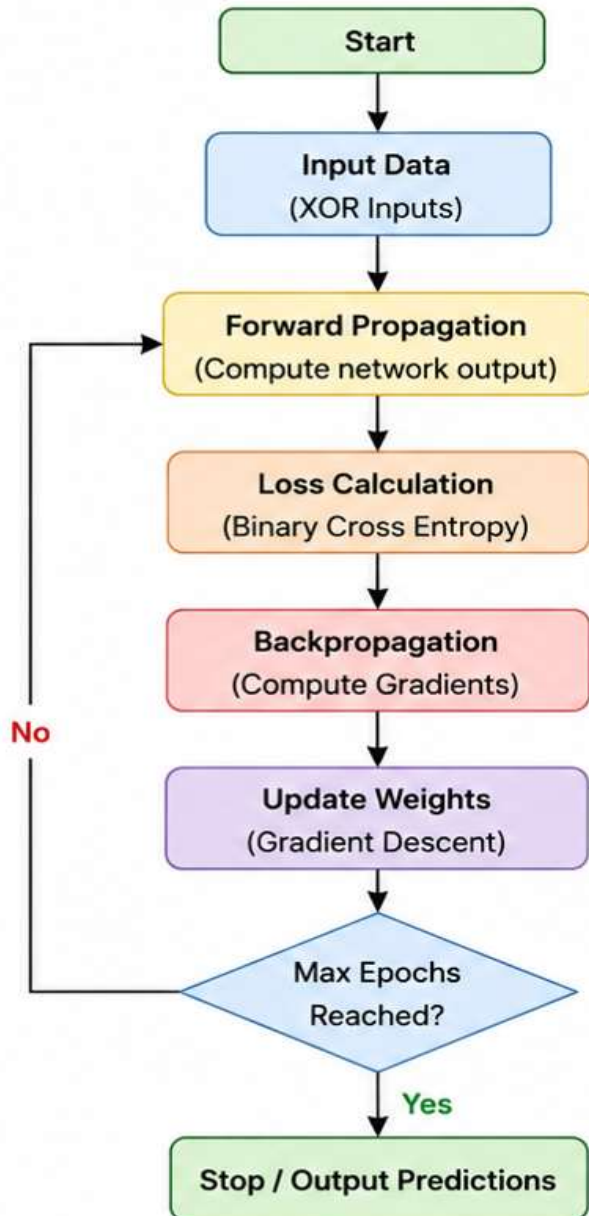
IMPLEMENTATION OF XOR USING MLP FROM SCRATCH

MLP Architecture for XOR



- **Input:** 2 features (x_1, x_2)
- **Hidden Layer 1:** 4 neurons with tanh activation
- **Hidden Layer 2:** 3 neurons with tanh activation
- **Output Layer:** 1 neuron with sigmoid activation

Working Flowchart (Training Process)



Training Process Explanation

1. Input Data

XOR dataset with two input features: x_1 and x_2 .
Target output is either **0** or **1**.

2. Forward Propagation

Inputs pass through the hidden layers.
Each neuron computes weighted sums and applies activation functions.
The network generates the predicted output.

3. Loss Calculation

Binary Cross Entropy (BCE) loss is computed.
Measures the difference between predicted and actual outputs.

4. Backpropagation

Gradients are calculated using the chain rule.
Error is propagated backward through the network.

5. Weight Update

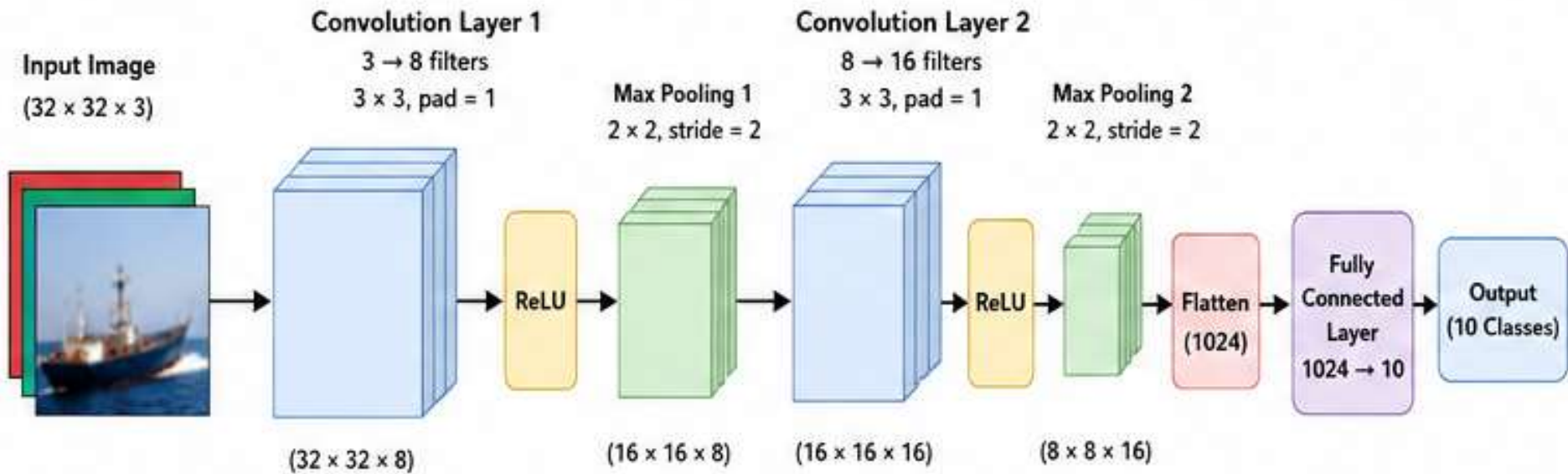
Weights and biases are updated using Gradient Descent.
This reduces the prediction error.

6. Repeat

Steps are repeated for **5000 epochs** until the loss converges.

Week 3 – Q1: CNN Forward & Backward Pass from Scratch

CNN ARCHITECTURE (Week 3 – Q1: From Scratch)



Convolution
(Feature Extraction)



ReLU
(Activation)



Max Pooling
(Downsampling)



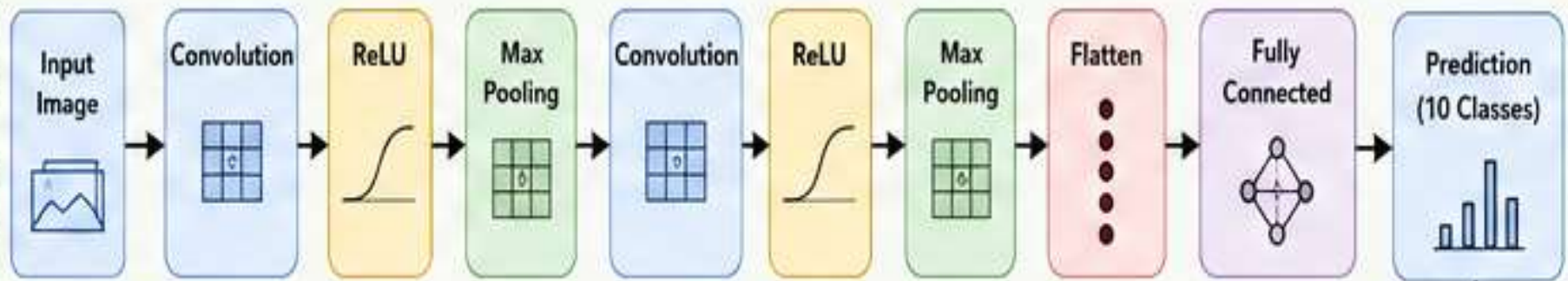
Flatten
(Feature Vector)



Fully Connected
(Classification)

WORKING FLOW (CNN – Forward & Backward Pass)

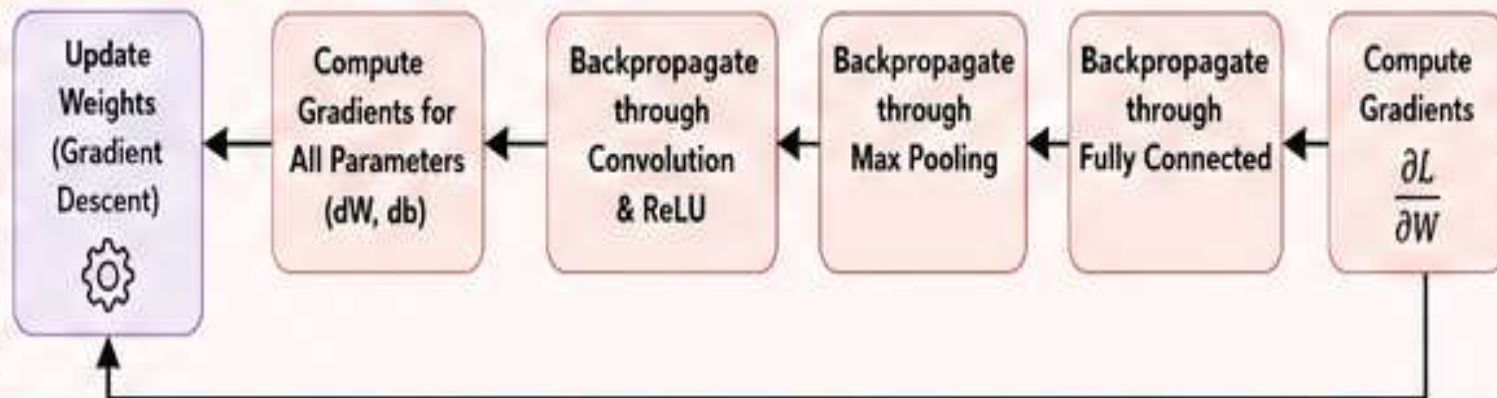
FORWARD PASS



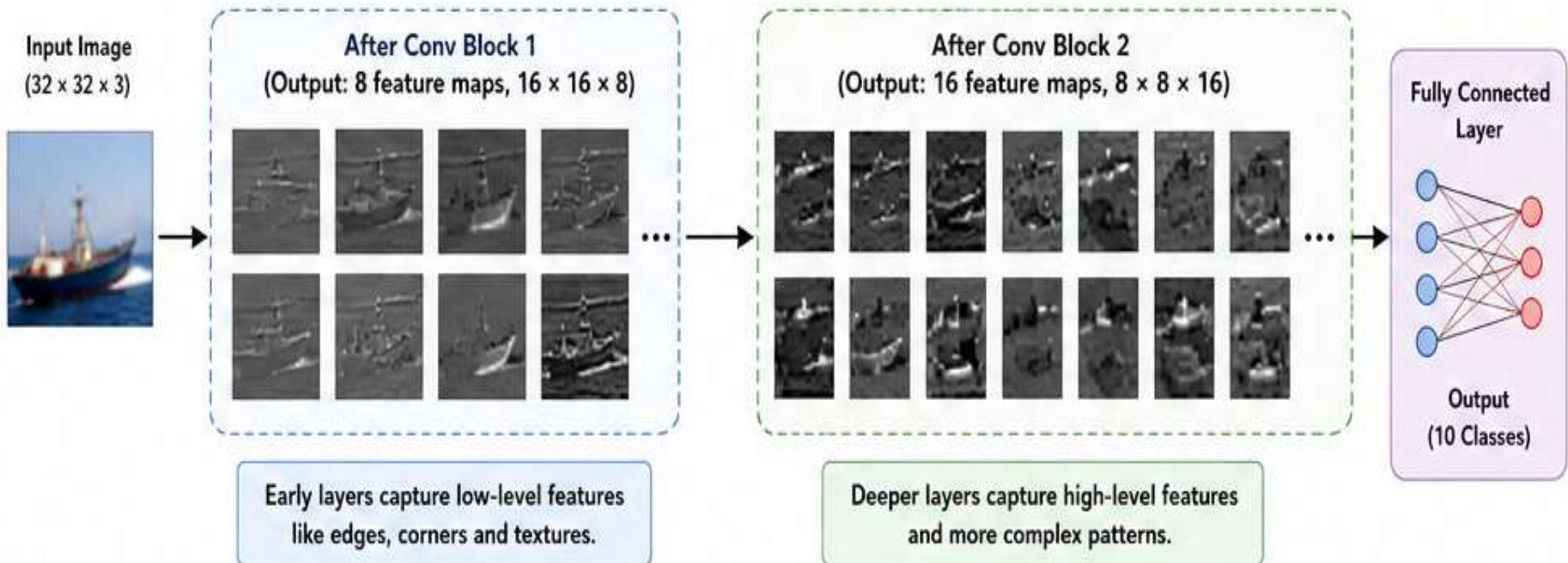
Loss
(Cross Entropy)



BACKWARD PASS (Backpropagation)



FEATURE MAP VISUALIZATION (Week 3 – Q2: PyTorch CNN)



Key Takeaways

- CNN learns hierarchical representations.
- Early layers detect simple patterns.
- Deeper layers learn complex and abstract features.
- These learned features help in accurate classification.

Decrease loss, increase test accuracy through multiple epochs (Model is learning)